Digital Image Analysis Tutorial 11

```python
#!/usr/bin/env python3
"""
AI531C — T11: The Training Studio
Lightweight linear image denoising: direct vs residual, noise mismatch,
median baseline, and optional range-training extension.

Run:
    python3 T11_studio.py
    python3 T11_studio.py support
    python3 T11_studio.py extension
    python3 T11_studio.py --image path/to/your_image.png

Dependencies:
    pip install numpy scipy scikit-image
"""

import argparse
from pathlib import Path

import numpy as np
from scipy.ndimage import median_filter
from skimage import data, io, color, img_as_float32
from skimage.metrics import peak_signal_noise_ratio


SEED = 11
PATCH_SIZE = 5
TRAIN_PATCHES = 12000
TEST_PATCHES = 6000


def load_image(image_path=None):
    """Load a grayscale image in the range [0, 1]."""
    if image_path:
        image = io.imread(image_path)
    else:
        image = data.camera()

    if image.ndim == 3:
```

```
        if image.shape[-1] == 4:
            image = image[..., :3]
        image = color.rgb2gray(image)

    image = img_as_float32(image)
    if image.ndim != 2:
        raise ValueError("Please provide a grayscale or RGB image.")
    if min(image.shape) < PATCH_SIZE + 2:
        raise ValueError("Image is too small for a 5x5 denoising window.")
    return np.clip(image, 0.0, 1.0)


def add_noise(image, sigma, rng):
    """Add Gaussian noise; sigma is expressed in 8-bit intensity units."""
    noisy = image + rng.normal(0.0, sigma / 255.0, image.shape)
    return np.clip(noisy, 0.0, 1.0)


def sample_patches(image, count, rng):
    """Sample 5x5 noisy-image windows and their clean centre pixels."""
    radius = PATCH_SIZE // 2
    rows = rng.integers(radius, image.shape[0] - radius, size=count)
    cols = rng.integers(radius, image.shape[1] - radius, size=count)
    features = np.empty((count, PATCH_SIZE * PATCH_SIZE), dtype=np.float64)
    targets = image[rows, cols].astype(np.float64)

    for i, (row, col) in enumerate(zip(rows, cols)):
        features[i] = image[
            row - radius:row + radius + 1,
            col - radius:col + radius + 1
        ].reshape(-1)
    return features, targets, rows, cols


def fit_ridge(features, targets, alpha=1e-5):
    """Fit a small ridge-regression model with an intercept."""
    x_mean = features.mean(axis=0)
    y_mean = targets.mean()
```

```
    x_centered = features - x_mean
    y_centered = targets - y_mean
    weights = np.linalg.solve(
        x_centered.T @ x_centered + alpha * np.eye(features.shape[1]),
        x_centered.T @ y_centered
    )
    bias = y_mean - x_mean @ weights
    return weights, bias


def predict(features, model):
    weights, bias = model
    return features @ weights + bias


def psnr(reference, estimate):
    estimate = np.clip(estimate, 0.0, 1.0)
    return float(peak_signal_noise_ratio(reference, estimate, data_range=1.0))


def make_train_data(clean, rng, count, sigma_range):
    sigma_values = rng.uniform(sigma_range[0], sigma_range[1], size=count)
    radius = PATCH_SIZE // 2
    rows = rng.integers(radius, clean.shape[0] - radius, size=count)
    cols = rng.integers(radius, clean.shape[1] - radius, size=count)
    x = np.empty((count, PATCH_SIZE * PATCH_SIZE), dtype=np.float64)
    y_clean = clean[rows, cols].astype(np.float64)
    y_noisy_center = np.empty(count, dtype=np.float64)

    # Use a separate noisy image per sigma group for a realistic training set.
    noisy_cache = {}
    for i, (row, col, sigma) in enumerate(zip(rows, cols, sigma_values)):
        key = round(float(sigma), 1)
        if key not in noisy_cache:
            noisy_cache[key] = add_noise(clean, key, rng)
        noisy = noisy_cache[key]
        x[i] = noisy[
            row - radius:row + radius + 1,
```

```
            col - radius:col + radius + 1
        ].reshape(-1)
        y_noisy_center[i] = noisy[row, col]
    return x, y_clean, y_noisy_center


def evaluate(clean, model, sigma, rng, label):
    noisy = add_noise(clean, sigma, rng)
    radius = PATCH_SIZE // 2
    # Evaluate the same valid interior for every method.
    windows = np.lib.stride_tricks.sliding_window_view(
        noisy, (PATCH_SIZE, PATCH_SIZE)
    )
    features = windows.reshape(-1, PATCH_SIZE * PATCH_SIZE)
    direct = np.clip(predict(features, model["direct"]), 0.0, 1.0)
    residual_noise = predict(features, model["residual"])
    noisy_centres = noisy[radius:-radius, radius:-radius].reshape(-1)
    residual = np.clip(noisy_centres - residual_noise, 0.0, 1.0)
    clean_valid = clean[radius:-radius, radius:-radius]

    median = median_filter(noisy, size=3)[radius:-radius, radius:-radius]
    direct_img = direct.reshape(clean_valid.shape)
    residual_img = residual.reshape(clean_valid.shape)

    direct_db = psnr(clean_valid, direct_img)
    residual_db = psnr(clean_valid, residual_img)
    median_db = psnr(clean_valid, median)

    print(f"\n{label} (test noise sigma = {sigma})")
    print("-" * 65)
    print(f"{'Method':<24}{'PSNR (dB)':>14}")
    print(f"{'Direct linear model':<24}{direct_db:>14.2f}")
    print(f"{'Residual linear model':<24}{residual_db:>14.2f}")
    print(f"{'3x3 median baseline':<24}{median_db:>14.2f}")
    print(f"Direct/residual gap: {abs(direct_db - residual_db):.4f} dB")
    return direct_db, residual_db, median_db
```

```python
def train_models(clean, train_sigma_range, rng, count=TRAIN_PATCHES):
    features, clean_targets, noisy_centres = make_train_data(
        clean, rng, count, train_sigma_range
    )
    noise_targets = noisy_centres - clean_targets

    direct_model = fit_ridge(features, clean_targets)
    residual_model = fit_ridge(features, noise_targets)
    return {"direct": direct_model, "residual": residual_model}

def main():
    parser = argparse.ArgumentParser(description="AI531C T11 denoising studio")
    parser.add_argument(
        "mode", nargs="?", choices=["support", "extension"],
        help="support: run the basic camera/direct/baseline cell; "
             "extension: train across sigma 8–30"
    )
    parser.add_argument(
        "--image", type=str, default=None,
        help="optional path to your own grayscale or RGB image"
    )
    parser.add_argument("--patches", type=int, default=TRAIN_PATCHES)
    args = parser.parse_args()

    rng = np.random.default_rng(SEED)
    clean = load_image(args.image)
    print("AI531C — T11: The Training Studio")
    print(f"Image: {args.image if args.image else 'skimage.data.camera()'}")
    print(f"Image size: {clean.shape[1]} x {clean.shape[0]}")
    print(f"Training patches: {args.patches}")
    print("Noise: additive Gaussian, clipped to [0, 1]")

    if args.mode == "support":
        train_range = (15.0, 15.0)
        models = train_models(clean, train_range, rng, args.patches)
        evaluate(clean, models, 15, rng, "SUPPORT: specialist model")
        print("\nSupport mode completed: specialist rows and median baseline only.")
        print("Note: measured values depend on the input image, sampling, and seed.")
        return
```

```python
    if args.mode == "extension":
        train_range = (8.0, 30.0)
        models = train_models(clean, train_range, rng, args.patches)
        row15 = evaluate(clean, models, 15, rng, "EXTENSION: range-trained model")
        row30 = evaluate(clean, models, 30, rng, "EXTENSION: range-trained model")
        print("\nREGIME TABLE (fill in from the printed measurements)")
        print(f"{'Training sigma':<22}{'Test sigma':<14}{'Direct (dB)':<15}{'Residual (dB)':<16}{'Median (dB)'}")
        print(f"{'8–30':<22}{'15':<14}{row15[0]:<15.2f}{row15[1]:<16.2f}{row15[2]:.2f}")
        print(f"{'8–30':<22}{'30':<14}{row30[0]:<15.2f}{row30[1]:<16.2f}{row30[2]:.2f}")
        print("\nTrade-off: range-training may slightly reduce specialist performance at")
        print("sigma 15 while improving robustness at sigma 30. Verify using your run.")
        return

    # Default: specialist training at sigma 15, then mismatch test at sigma 30.
    models = train_models(clean, (15.0, 15.0), rng, args.patches)
    row15 = evaluate(clean, models, 15, rng, "SPECIALIST: matched test")
    row30 = evaluate(clean, models, 30, rng, "MISMATCH: test sigma differs from training")
    print("\nRESULTS TABLE")
    print(f"{'Training sigma':<22}{'Test sigma':<14}{'Direct (dB)':<15}{'Residual (dB)':<16}{'Median (dB)'}")
    print(f"{'15':<22}{'15':<14}{row15[0]:<15.2f}{row15[1]:<16.2f}{row15[2]:.2f}")
    print(f"{'15':<22}{'30':<14}{row30[0]:<15.2f}{row30[1]:<16.2f}{row30[2]:.2f}")
    print("\nRequired reporting sentence: Results must state the training-noise range")
    print("and test noise sigma; performance at sigma 15 does not imply performance")
    print("at sigma 30.")
    print("\nFor your lane's data, rerun with --image path/to/image.png.")


if __name__ == "__main__":
    main()
```

Output:

```
$ python T11_studio.py --image input2.png
AI531C — T11: The Training Studio
Image: input2.png
Image size: 754 x 425
Training patches: 12000
Noise: additive Gaussian, clipped to [0, 1]

SPECIALIST: matched test (test noise sigma = 15)
----------------------------------------------------------------
Method                        PSNR (dB)
Direct linear model               29.94
Residual linear model             29.94
3x3 median baseline               22.82
Direct/residual gap: 0.0000 dB

MISMATCH: test sigma differs from training (test noise sigma = 30)
----------------------------------------------------------------
Method                        PSNR (dB)
Direct linear model               24.40
Residual linear model             24.40
3x3 median baseline               21.84
Direct/residual gap: 0.0000 dB

RESULTS TABLE
Training sigma        Test sigma    Direct (dB)    Residual (dB)   Median (dB)
15                    15            29.94          29.94           22.82
15                    30            24.40          24.40           21.84

Required reporting sentence: Results must state the training-noise range
and test noise sigma; performance at sigma 15 does not imply performance
at sigma 30.

For your lane's data, rerun with --image path/to/image.png.
```